

**Programación**Equipo AFIC (ayudantias.fic@org.estudiantesudp.cl)**1) La servilleta de María.**

María estaba tomando asistencia en una ayudantía cuando Kanvas decidió esconder el módulo, cerrar sesión y mirar para el lado como si no fuera con él. Como la universidad, en su infinita coordinación tecnológica, no siempre coopera, María terminó anotando la asistencia en una servilleta.

La lista indica si un mechón asistió a cada una de las 7 ayudantías:

```
[1, 1, 0, 1, 0, 1, 1]
```

Programe:

- `contar_asistencias(lista);`
- `calcular_porcentaje(lista);`
- `estado_asistencia(lista)`, que retorne "regular" si el porcentaje es al menos 75% y "revisar" en caso contrario.

2) La economía inexplicable de la JUNA.

Miki fue a comprar almuerzo con 2000 pesos, que en teoría alcanza para algo, salvo que uno intente comprar un completo en la U y descubra que la arepa, por alguna razón misteriosa, es más barata. No vamos a resolver el modelo económico del casino, solo vamos a ordenar los datos.

Use la siguiente lista de tuplas:

Producto	Precio
completo	3100
arepa	2600
empanada	1800
agua	900
café	600
galleta	750

Programe:

- `producto_mas_barato(productos);`
- `producto_mas_caro(productos);`
- `promedio_precios(productos);`
- `puede_comprar(productos, monto).`

La última función debe retornar todos los productos que no superen el monto recibido.



3) Una alcancía fantástica.

Leo apareció diciendo que tenía una alcancía fantástica. Uno esperaba una maravilla financiera, tal vez una máquina con luces, tal vez una inversión de alto nivel. Era una alcancía normal, pero con una etiqueta que decía “no abrir hasta la solemne”, lo cual igual le da cierto respeto.

Sus depósitos diarios fueron:

[500, 1000, 0, 700, 1200, 300, 800]

Programe funciones para calcular:

- total ahorrado;
- promedio diario;
- día con mayor depósito;
- cantidad de días sobre el promedio;
- si llegó o no a una meta de 5000 pesos.

4) Web Dev

Fernando ha dado Desarrollo web y Optimización tantas veces que ya no se sabe si son ramos, estaciones del año o un ciclo natural de la facultad. Cada semestre alguien dice “ahora sí será distinto”, y ahí está Fernando de nuevo, viendo aparecer otro formulario, otro simplex y otro HTML con emojis.

Cree una lista de tuplas con el formato:

(ramo, semestre)

Incluya al menos 10 registros usando "Desarrollo Web", "Optimización" y otros ramos. Luego programe funciones para:

- contar cuántas veces aparece cada ramo;
- obtener el ramo más repetido;
- listar los semestres en que aparece "Desarrollo Web";
- calcular qué porcentaje de los registros corresponde a "Optimización".

5) Observación administrativa

El ayudante E no será juzgado en esta guía. Eso sería demasiado trabajo y además habría que hacer pauta. Solo será medido, que es peor, porque los datos no se cansan. La lista muestra cuántas veces lo retaron por decir algo demasiado intenso durante 6 semanas:

[2, 0, 1, 3, 1, 4]

Programe funciones para calcular:

- total de retos;
- promedio semanal;
- semana con más retos;
- semanas que estuvieron sobre el promedio;
- clasificación final: "tranquilo", "observado" o "reunión con coordinación".

**6) El retorno de C**

El ayudante C tiene una situación estadísticamente sospechosa, muchos le caen mal, pero cuando no está todos preguntan cuándo vuelve. En especial los de primer año. Uno mira los datos y piensa que esto no es cariño, pero tampoco es rechazo. Es algo peor, dependencia académica.

Use una lista de tuplas con:

```
(generacion, reclamos, personas_pidiendo_retorno)
```

Programa funciones para:

- calcular el total de reclamos;
- calcular el total de personas pidiendo retorno;
- retornar las generaciones donde los pedidos de retorno superan los reclamos;
- construir un diccionario donde la llave sea la generación y el valor sea "vuelve" o "mejor esperar".

7) Notas mechas.

Una sección de primer semestre rindió 4 controles chicos. No hay destino, no hay maldición, no hay “es que python me odia”. Hay una tabla, números y la obligación mínima de calcular antes de opinar.

Estudiante	C1	C2	C3	C4
mechón 1	5.5	4.8	6.0	5.9
mechón 2	3.2	4.0	3.8	4.1
mechón 3	6.1	5.9	6.4	6.0
mechón 4	2.8	3.5	4.2	3.9

Guarde los datos en una lista de listas o lista de tuplas. Programe:

- `promedio_estudiante(fila);`
- `promedios_curso(datos);`
- `estudiantes_en_riesgo(datos), considerando riesgo bajo 4.0;`
- `mejor_promedio(datos).`

8) The JUNA files.

Después de discutir demasiado sobre por qué una arepa cuesta menos que un completo, AFIC decidió hacer lo único razonable, guardar los datos en un archivo y obligar a python a mirarlos con calma. Lea `datos/juna.csv`. El archivo tiene encabezado y columnas separadas por ; :

```
local;producto;precio;categoria
```

Construya un programa que:

- lea el archivo ignorando el encabezado;
- guarde cada fila en una lista;
- cree un diccionario donde la llave sea el local y el valor sea una lista de productos;
- muestre el producto más barato;
- cuente cuántos productos hay por categoría.

**9) Asistencia.**

Hay estudiantes que aseguran haber ido “a casi todas” las ayudantías. Esa frase suele significar cualquier cosa entre 2 y 99, dependiendo de la memoria, el sueño y la carrera (jeje). Para evitar filosofía, lea `datos/asistencia_mechona.csv`. Cada fila contiene:

```
nombre;seccion;semanal1;semana2;semana3;semana4;semana5;semana6
```

Construya un diccionario donde la llave sea el nombre y el valor sea otro diccionario con sección, total de asistencias y porcentaje. Luego muestre estudiantes bajo 75%, promedio de asistencia por sección y sección con mejor asistencia.

10) Veterinaria.

En veterinaria no sirve una planilla preciosa si al final nadie sabe qué paciente falta por atender. Lea `datos/veterinaria.csv`. Cada fila contiene un animal, su especie, peso, nivel de urgencia y si fue atendido.

Programe funciones para:

- leer el archivo como lista de diccionarios;
- contar pacientes por especie;
- calcular peso promedio por especie;
- mostrar pacientes con urgencia mayor o igual a 4;
- contar pacientes pendientes.

11) El cable de Robótica UDP.

La comunidad de robótica puede tener sensores, motores y toda la voluntad del mundo, pero siempre falta un cable. Siempre. Alguien lo vio, alguien lo prestó, alguien jura que estaba en la caja y al final el robot queda mirando al vacío como proyecto de feria sin presupuesto emocional.

Lea `datos/robotica.csv`, con columnas:

```
equipo;pieza;cantidad;estado
```

Construya:

- una lista con todas las filas;
- un diccionario donde la llave sea la pieza y el valor sea la cantidad total;
- un set con los estados distintos;
- una función que reciba una pieza y diga si alcanza para armar 3 robots, considerando que cada robot necesita 2 unidades de esa pieza.

12) El sueño de n!.

La comunidad $n!$ organizó una actividad tranquila. Tranquila en teoría. Primero había 5 inscritos, después cada inscrito invitó a alguien, después alguien invitó a alguien que no sabía de qué era la actividad y al final el guardia preguntó por qué había tanta gente tan emocionada. (Era un sueño de la directiva).

Lea `datos/nfactorial.csv`:

```
nombre;carrera;invitado_por
```

Calcule inscritos por carrera, personas invitadas por cada participante, carrera con más inscritos y nombres ordenados alfabéticamente.

**13) Metal Guar Sólido contra Nier Automático.**

Edicson insiste en que *Metal Guar Sólido* es la obra más importante desde que alguien inventó guardar partida. Cristopher responde que *Nier Automático* tiene más alma y lo defiende con energía de huaso protegiendo su parcela. Nadie pidió esta guerra, pero Steam dejó datos.

Lea datos/steam.csv:

```
jugador;juego;minutos;semana
```

El programa debe calcular minutos totales por juego, minutos totales por jugador, juego más jugado, jugadores con más de 500 minutos y horas jugadas redondeadas a 2 decimales.

Desafío: haga un gráfico de barras con los minutos totales por juego.

14) Santiago no es Chile.

Cristopher, pensando en los caminos de tierra, afirma que cualquier problema de transporte se arregla “saliendo antes”, una frase muy bonita hasta que uno depende de micro, taco, lluvia y una sala cambiada 10 minutos antes. Use la siguiente lista de tuplas:

```
(medio, minutos, atraso)
```

Cree al menos 8 registros. `atraso` debe ser 1 si llegó tarde y 0 si llegó a tiempo. Programe funciones para calcular tiempo promedio por medio de transporte, cantidad de atrasos por medio y el medio con mayor atraso acumulado.

15) Voluntarios AFIC y la lista que nunca está lista.

Para una actividad de AFIC, Miki dijo “yo veo logística”, María dijo “yo apoyo difusión”, Leo apareció cuidando la alcancía fantástica, Fernando terminó revisando una entrega de su página, Edicson fue a tirarle piedras a la escuela de industria y Cristopher dijo que podía ayudar siempre que hubieran vaquitas y ovejas. Todo normal.

Lea datos/voluntarios_afic.csv:

```
nombre;area;horas;confirmado
```

Cuente voluntarios por área, sume horas confirmadas, muestre quienes no han confirmado y cree un set con las áreas distintas.

16) Este año no hay ayudantía.

Hay un ayudante A de la U que cancela ayudantías con una constancia admirable. Uno ya no sabe si está ocupado, si vive lejos, si pelea contra Kanvas o si simplemente está entrenando para desaparecer con elegancia. Lea datos/cancelaciones_ayudantia.csv:

```
ayudante;semana;motivo;reprogramada
```

Construya un diccionario con total de cancelaciones por ayudante, cuente cuántas fueron reprogramadas y muestre el ayudante con más cancelaciones. Si hay empate, muestre todos los empatados.

**17) Detector.**

Un grupo de estudiantes entregó tareas con comentarios como “Claro, aquí tienes, esta función optimiza de manera eficiente el paradigma de datos”. Nadie habla así después de almorzar en la U. Para no acusar por intuición, se guardaron señales en `datos/entregas_ia.csv`:

```
estudiante;lineas;frases_sospechosas;comentarios_raros
```

Calcule:

```
puntaje = frases_sospechosas * 2 + comentarios_raros
```

Muestre estudiantes con puntaje mayor o igual a 5, promedio de puntaje y estudiante con mayor puntaje. Esto no acusa a nadie, solo ordena señales. La explicación del código sigue siendo responsabilidad humana.

18) Alias.

Como nadie quiere escribir *Metal Guar Sólido* completo veinte veces, AFIC armó un archivo de alias. Lea el archivo:

```
datos/alias_afic.csv
```

```
alias;significado
```

Cargue el archivo en un diccionario y úselo para traducir:

```
["MGS", "NIA", "JUNA", "AFIC", "AYC", "ALC"]
```

Si un alias no existe, muestre `"alias desconocido"`.

19) El guardia de la U y el reporte de patio.

El guardia escuchó demasiado: gente hablando de la juna, alguien diciendo que K4st programaría en excel metafóricamente, primer año preguntando por el retorno del ayudante C y una discusión absurda sobre si *Nier Automático* cuenta como ramo humanista. En vez de sufrirlo en silencio, dejó un archivo.

Lea `datos/reporte_patio.txt`:

```
tema;persona;cantidad
```

Programe lectura del archivo, diccionario con total por tema, persona que más aparece acumulando cantidades y lista de temas ordenados de mayor a menor frecuencia.

20) Informe final.

Ahora mezcle archivos. Use `datos/asistencia_mechona.csv`, `datos/notas_control.csv` y `datos/entregas_ia.csv`. La coordinación quiere un resumen, no una novela de 400 líneas ni una función llamada `cosa`.

El informe debe mostrar:

- promedio de notas por estudiante;
- porcentaje de asistencia por estudiante;
- puntaje de tarea con olor raro por estudiante;
- clasificación final: `"sin alerta"`, `"revisar asistencia"`, `"revisar notas"` o `"revisar entrega"`.

Debe existir una función principal que llame a funciones auxiliares para calcular promedio, asistencia y puntaje.



Desafío final: haga un gráfico de barras con el promedio de notas por estudiante. Si los nombres quedan montados unos sobre otros, arregle el gráfico antes de decir que “en mi compu se veía bien”.

21) **BuscaMinas.**

Debes implementar una versión simplificada del juego **Buscaminas**. El juego se desarrolla sobre un tablero de 5×5 casillas. El programa debe:

1. Generar el tablero de forma aleatoria al iniciar, colocando las bombas en posiciones distintas.
2. Preguntar por alguna casilla hasta que el usuario ingrese el número -1 o se pierda al caer en una bomba.
3. Si la casilla elegida contiene una bomba $\rightarrow$ mostrar el mensaje " ¡ GAME OVER ! " y terminar el juego.
4. Si la casilla no contiene bomba $\rightarrow$ mostrar cuántas bombas hay en las **casillas vecinas** (incluyendo diagonales) y continuar permitiendo nuevas elecciones hasta que se encuentre una bomba o se ingrese -1.

Matriz de ejemplo

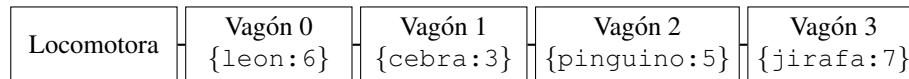
A continuación se muestra una configuración inicial posible del tablero (las bombas están representadas por **X** y las casillas vacías por **0**).

0	0	X	0	X
0	X	0	0	0
0	0	0	X	0
0	0	0	0	0
X	0	X	0	0



- 22) **La locomotora** Finalmente se sabe el paradero del ayudante A y el porque de sus desapariciones, se le vio en un tren rumbo a su hogar, el problema es que no ha vuelto desde entonces, lo unico que se sabe es que era un tren de animales y el no podia entrar a un vagón donde hubieran más de 5 animales, lo cual explicaria su ausencia.

Representación gráfica del tren



Desarrollo

Debe desarrollar un programa que permita al usuario interactuar con el tren mediante un menú de opciones. El programa debe:

- Mostrar un menú con las siguientes opciones:
 - Agregar animales** a un vagón (pedir número de vagón, nombre del animal y cantidad).
 - Mostrar todo el tren** (listar el contenido de cada vagón).
 - Buscar un animal** en todo el tren (indicar en qué vagón(es) y cuántos hay).
 - Transferir animales** de un vagón a otro (especificar origen, destino, animal y cantidad).
 - Calcular total de animales** por especie en todo el tren.
 - Ver si el ayudante viene en el tren:** Mostrar por pantalla si el ayudante pudo subir al tren o no, en que vagón y con que animales va (siempre sube al con menor cantidad).
 - Salir** del programa.
- El programa debe ejecutarse en un bucle hasta que el usuario elija salir.
- Validar todas las entradas (que el vagón exista, que el animal exista cuando se requiera, que las cantidades sean positivas y no excedan lo disponible, etc.).
- Mostrar mensajes claros de error o éxito en cada operación.



- 23) **Depuración: Temperaturas.** El siguiente código pretende leer 24 temperaturas, calcular máxima, mínima y promedio, y luego consultar una hora específica. Sin embargo, contiene varios errores. Encuéntrelos y corrijalos.

```
temperaturas = []

# Pedir temperatura para cada una de las 24 horas
for hora in range(24)
    temp = input(f"Ingrese la temperatura de la hora {hora+1}: ")
    temperaturas.append(temp)

max_temp = 0
min_temp = 0

for temp in temperaturas:
    if temp > max_temp:
        max_temp = temp
    if temp < min_temp:
        min_temp = temp

promedio = sum(temperaturas) / len(temperaturas)
promedio = round(promedio)

print(f"Temperatura más alta: {max_temp}")
print(f"Temperatura más baja: {min_temp}")
print(f"Temperatura promedio: {promedio}")

hora = int(input("Ingrese una hora del día para ver su temperatura: "))
print(temperaturas[hora-1])
```

- 24) **Depuración: Notas y gráfico.** Este programa lee un archivo data/notas.csv (con columnas nombre, asignatura, nota), calcula el promedio de cada estudiante (asumiendo que tienen exactamente 3 notas) y muestra un gráfico de barras. Tiene errores de sintaxis, lógica y de manejo de archivos. Corrijalos.

```
import csv
import matplotlib.pyplot as plt

datos = []
with open("data/notas.csv", "r", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f, delimiter=",")
    for i in reader:
        datos.append(i)

sumNotas= {}

for i in datos:
    nombre = i["nombre"]
    nota = float(i["nota"])
    if nombre not in sumNotas:
        sumNotas[nombre] = nota
    else:
        sumNotas[nombre] += nota
```



```
promedio = []
nombres = []

for nombre, notas in sumNotas.items():
    print(f"===== {nombre} =====")
    for i in datos:
        if nombre == i["nombre"]:
            print(f"Asignatura: {i['asignatura']}, nota: {i['nota']}")
    print(f"tiene un promedio de: {round(notas/3,1)}")
    nombres.append(nombre)
    promedio.append(round(notas/3,1))

plt.bar(nombres, promedio)
plt.title("Promedios de los alumnos")
plt.show()
```

- 25) **Depuración: Carrera de Metapods.** El siguiente código simula una carrera de tres Metapods. Cada turno avanzan una cantidad aleatoria entre 1 y 3. El primero en llegar a 15 o más gana. Pero la lógica de la condición de término y algunos detalles están mal. Corrija los errores.

```
import random

metapod1 = 0
metapod2 = 0
metapod3 = 0
i=1

while metapod1 < 15 or metapod2 < 15 or metapod3 < 15:
    print("TURNO : "+ str(i))
    metapod1 += random.randint(1, 3)
    metapod2 += random.randint(1, 3)
    metapod3 += random.randint(1, 3)

    print("Metapod 1: " +str(metapod1) + "bloques de distancia")
    print("Metapod 2: " +str(metapod2) + "bloques de distancia")
    print("Metapod 3: " +str(metapod3) + "bloques de distancia")
    i+=1
    print(f"=====")

if metapod1 > metapod2 and metapod1 > metapod3:
    print("El ganador es el metapod 1")
elif metapod2 > metapod1 and metapod2 > metapod3:
    print("El ganador es el metapod 2")
else:
    print("El ganador es el metapod 3")
```



- 26) **Depuración: Clasificación de hongos.** Este programa clasifica hongos como comestibles, venenosos o desconocidos según ciertos atributos, y calcula un precio. Tiene errores en las condiciones lógicas y en los cálculos. Encuéntrelos y corrija los.

```
forma = int(input("Ingrese la forma del sombrero:"))
color = int(input("Ingrese el color del sombrero:"))
adherencia = int(input("Ingrese la adherencia de las láminas:"))
anillo = int(input("Ingrese si presenta anillo:"))
volva = int(input("Ingrese si presenta volva:"))
color_espora = int(input("Ingrese el color de la espora:"))
olor = int(input("Ingrese el olor del hongo:"))
peso = int(input("Ingrese el peso en gramos:"))

if volva == 1 and anillo == 1:
    if color == 2 or 3 or 4:
        print("El hongo es venenoso")
    elif color == 1 and color_espora == 1:
        print("El hongo es comestible")
    else:
        print("El hongo es desconocido")
    precio = ( pow(peso, 2)) * 5 + color * 100 ) / 3
    print("El precio del hongo es de: ", precio)

elif volva == 1 and anillo == 0:
    print("El hongo es desconocido")
    precio = peso * forma * 2 + 50
    print("El precio es: ", precio)

elif volva == 0 and anillo == 1:
    if adherencia == 1 and color_espora == 4:
        print("El hongo es comestible")
    elif color == 2 and olor == 3:
        print("El hongo es venenoso")
    else:
        print("El hongo es desconocido")
    precio = ( peso * (color_espora + adherencia) ) / 2
    print("El precio es: ", precio)

else:
    if forma == 1 and olor == 5:
        print("El hongo es comestible")
    elif color == 5 and adherencia == 1:
        print("El hongo es venenoso")
    else:
        print("El hongo es desconocido")
    precio = (peso * color) * (forma + 3)
    print("El precio es: ", precio)
```



- 27) **Depuración: Acceso al club.** El siguiente código determina si una persona puede entrar a un club según su edad, membresía, invitación o acompañante. Tiene errores en la lógica y en el manejo de entradas. Corríjalos.

```
edad = int(input("Ingrese su edad: "))

if edad >= 18:
    membresia = input("tiene membresia:")
    invitacion = input("Tiene invitación:")

    if membresia or invitacion:
        print("Usted puede entrar al club")
    else:
        print("Usted no puede ingresar al club")

elif edad >= 12 and edad <= 17:
    acompañado = input("Vienes acompañado:")
    if acompañado:
        print("Usted puede entrar al club")
    else:
        print("Usted no puede ingresar al club")
else:
    print("Usted no puede ingresar al club")
```

*“No tengo ni idea de adónde nos llevará esto,
pero tengo la clara sensación de que será un
lugar maravilloso y extraño a la vez.”*

— Dale Cooper, *Twin Peaks*